Module 1 · Topic 1: JVM Memory & Garbage Collection

Java, Spring Boot, Microservices & Cloud-Native Backend Stack: Learning Notes

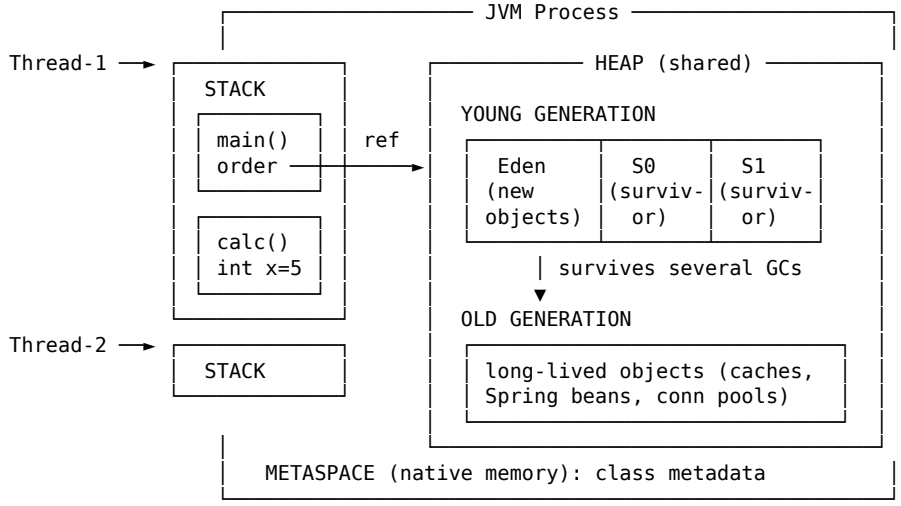
1. Analogy: A Restaurant Kitchen

Think of the JVM as a busy restaurant kitchen:

Kitchen	JVM
Each **chef's personal cutting board**: small, private, wiped clean after each dish	**Stack**: one per thread. Holds method calls and local variables, freed automatically when a method returns
The **shared pantry** every chef stores ingredients in	**Heap**: shared by all threads. Every object you create with `new` lives here
The **recipe book** on the wall	**Metaspace**: class definitions, method code, static info
The **cleaning crew** that throws out ingredients nobody is using	**Garbage Collector (GC)**

The cleaning crew can't throw away anything a chef is still using. It only removes items that **no one can reach anymore**.

2. Architecture Diagram



What goes where?

```
public class OrderService {

    private static final double TAX_RATE = 0.18;   // Metaspace (class-level static info)

    public Order createOrder(String customerId) {   // new stack frame pushed for this call
        int quantity = 2;                             // STACK: primitive local, value itself
        Order order = new Order(customerId);          // `order` reference -> STACK
                                                      // the Order object  -> HEAP (Eden)
        return order;
    }                                                 // frame popped; object stays on heap
}                                                     // as long as something references it
```

Key rule: primitives and references live on the stack. **Objects always live on the heap.**

3. How Garbage Collection Works

Step 1: Find what's alive (reachability)

The GC starts from **GC Roots** (local variables on stacks, static fields, active threads) and follows every reference it can. Anything it can't reach is garbage.

```
GC Roots ──► Order ──► Customer        ✔ alive
             Cart  ──► Item            ✘ unreachable → garbage
```

Step 2: The generational hypothesis

Most objects die young, like request DTOs, temporary strings, and loop variables. So the heap is split up:

1. New objects go into **Eden**.
2. When Eden fills up → **Minor GC** (fast). Survivors move to S0/S1.
3. Objects that survive several cycles get **promoted** to **Old Gen**.
4. When Old Gen fills up → **Major/Full GC** (slower, can pause your app).

Step 3: Choosing a collector (Java 17+ / 21)

Collector	Best for	Flag
G1 (default)	Most Spring Boot services; balances throughput and pause time	`-XX:+UseG1GC`
ZGC	Low-latency APIs (pauses under 1 ms, even on huge heaps)	`-XX:+UseZGC` `-XX:+ZGenerational` (Java 21)
Parallel	Batch jobs where throughput matters more than pauses	`-XX:+UseParallelGC`

Production tuning example

```
java -Xms512m -Xmx512m \                # equal min/max avoids resize churn
     -XX:+UseG1GC \
     -XX:MaxGCPauseMillis=200 \
     -XX:+HeapDumpOnOutOfMemoryError \  # get evidence when things go wrong
     -Xlog:gc*:file=gc.log \
     -jar order-service.jar
```

4. Common Beginner Mistake: "Java has GC, so I can't leak memory"

You can. GC only frees **unreachable** objects. If you keep a reference by accident, the object lives forever:

```
@Service
public class SessionTracker {
    // BAD: static map grows forever - every entry is reachable from a GC Root
    private static final Map<String, UserSession> SESSIONS = new HashMap<>();

    public void login(String userId) {
        SESSIONS.put(userId, new UserSession(userId));
        // never removed -> eventually java.lang.OutOfMemoryError: Java heap space
    }
}
```

Fix: remove entries on logout, use a bounded cache (Caffeine with `maximumSize` / `expireAfterAccess`), or use an external store like Redis (covered in Module 4).

5. Check-in Question

Look at this code:

```
public void processOrders() {
    for (int i = 0; i < 1_000_000; i++) {
        Order temp = new Order("cust-" + i);
        temp.validate();
    }
}
```

Q1. Where does the variable `temp` live, and where do the `Order` objects live?

Q2. After each loop iteration, is the previous `Order` object eligible for garbage collection? Why or why not?

Q3. Will these million objects most likely be cleaned up by a Minor GC or a Major GC? Explain using the generational hypothesis.

Write down your answers, then share them in chat to move on to Topic 2: Java Collections internals.